

01_raw_documentLoader.py

```
from pathlib import Path

file_path = Path("./data/Sample_Text.txt")

with file_path.open("r", encoding="utf-8") as f:
    text = f.read()

print("--- Raw Python 결과 ---")
print(text)
print(type(text))

# 실습 시 주석은 해제해주세요.
# raw_doc = {
#     "page_content": text,
#     "metadata": {
#         "source": str(file_path),
#         "file_name": file_path.name,
#         "file_type": file_path.suffix,
#     },
# }

# print("\n--- 직접 만든 raw_doc ---")
# print(raw_doc["page_content"][:300])
# print(raw_doc["metadata"])
```

02_DocumentLoader_TextLoader.py

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("./data/Sample_Text.txt", encoding="utf-8")
docs = loader.load()

print("--- TextLoader 결과 ---")
print(f"메타데이터: {docs[0].metadata}\n")
print("--- 내용 ---")
print(docs[0].page_content)

print("docs type: ", type(docs))
print("doc type: ", type(docs[0]))
print("page_content: ", docs[0].page_content[:300])
print("metadata: ", docs[0].metadata)
```

03_DocumentLoader_CSVLoader.py

```
from langchain_community.document_loaders import CSVLoader
```

```
loader = CSVLoader(file_path="./data/Sample_CSV.csv", encoding="utf-8")
```

```
rows = loader.load()
```

```
print(f"--- CSVLoader 결과: 총 {len(rows)}개 Document ---")
```

```
for i, row in enumerate(rows):
```

```
    print("=" * 50)
```

```
    print(f"row {i}")
```

```
    print("metadata:", row.metadata)
```

```
    print("page_content:")
```

```
    print(row.page_content)
```

04_DocumentLoader_PyPDFLoader.py

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("./data/SPRi AI  
Brief_10월호_산업동향_1002_F.pdf")
pages = loader.load()

print("--- PyPDFLoader 결과 (첫 페이지) ---")
print(f"메타데이터: {pages[0].metadata}\n")
print("--- 내용 ---")
print(pages[10].page_content)

for i, page in enumerate(pages[:3]):
    print("=" * 50)
    print(f"페이지 인덱스: {i}")
    print(page.metadata)
    print(page.page_content[:300])
```

04-02_DocumentLoader_PyMuPDF

04-02_DocumentLoader_PyMuPDF.py

```
# pip install pymupdf

from langchain_community.document_loaders import PyMuPDFLoader
from pathlib import Path

pdf_filepath = Path("./data/000660_SK_2023.pdf")

loader = PyMuPDFLoader(pdf_filepath)
pages = loader.load()

print(len(pages))
print(pages[0].page_content)
print(pages[0].metadata)
```

05_DocumentLoader_WebBaseLoader.py

```
from langchain_community.document_loaders import WebBaseLoader

loader =
WebBaseLoader("https://n.news.naver.com/article/094/0000012995",
               bs_get_text_kwargs={"separator": " ", "strip": True})
docs = loader.load()

print("--- WebBaseLoader 결과 ---")
print(f"메타데이터: {docs[0].metadata}\n")
print("--- 내용 ---")
print(docs[0].page_content[:1000])
```

06_raw_textSplitter.py

```
text = """
LangChain은 LLM 애플리케이션 개발을 위한 프레임워크입니다.
```

이 프레임워크의 핵심은 모듈화입니다.

```
주요 기능으로는 모델 I/O, Chains, RAG 등이 있습니다.
"""
```

```
chunk_size = 50
chunks = []
```

```
for i in range(0, len(text), chunk_size):
    chunk = text[i:i + chunk_size]
    chunks.append(chunk)
```

```
print(f"전체 글자 수: {len(text)}")
print(f"chunk_size: {chunk_size}")
print(f"chunk 개수: {len(chunks)}")
```

```
for i, chunk in enumerate(chunks):
    print(f"[{i+1}] {chunk}")
```

07_Text Splitter_CharacterTextSplitter.py

```
from langchain_text_splitters import CharacterTextSplitter

sample_text = "안녕하세요. 반갑습니다.\n\nLangChain 강의에 오신  
것을 환영합니다.\n\n텍스트 분할을 알아보시다."

# separator: 분할 기준으로 사용할 문자
text_splitter = CharacterTextSplitter(
    separator="\n\n",
    chunk_size=50,
    chunk_overlap=10
)

chunks = text_splitter.split_text(sample_text)

print(f"--- CharacterTextSplitter (총 {len(chunks)}개) ---")
for i, chunk in enumerate(chunks):
    print(f"[{i+1}] {chunk}")
```


08_Text Splitter_RecursiveCharacterTextSplitter.py

```
from langchain_text_splitters import RecursiveCharacterTextSplitter
```

```
sample_text = """
Discourse에서 복사해주세요.
"""
```

```
for size in [50, 100, 200]:
    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=size,
        chunk_overlap=int(size * 0.2),
        separators=["\n\n", "\n", ". ", " ", ""],
    )

    chunks = text_splitter.split_text(sample_text)

    print("=" * 70)
    print(f"chunk_size={size}, chunk_overlap={int(size * 0.2)},
chunk_count={len(chunks)}")

    for i, chunk in enumerate(chunks):
        print(f"{i+1}번 청크, 길이={len(chunk)}")
        print(repr(chunk))
        print()
```

09_Text Splitter_RecursiveCharacterTextSplitter_pdf.py

```
from pathlib import Path
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter

# PDF 경로 설정
spri_pdf = Path("./data/SPRi AI Brief_10월호_산업동향_1002_F.pdf")
sample_pdf = Path("./data/Sample_RAG_Knowledge_Base.pdf")

pdf_path = spri_pdf if spri_pdf.exists() else sample_pdf

if not pdf_path.exists():
    raise FileNotFoundError("PDF 파일을 찾을 수 없습니다. ./data 폴더를 확인하세요.")

# 1. PDF 로드
loader = PyPDFLoader(str(pdf_path))
pages = loader.load()

print(f"로드한 PDF: {pdf_path.name}")
print(f"로드된 페이지 수: {len(pages)}")
print()

# 2. chunk_size를 바꿔가며 비교
for size in [500, 1000]:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=size,
        chunk_overlap=int(size * 0.1),
        separators=["\n\n", "\n", ". ", " ", ""],
    )

    chunks = splitter.split_documents(pages)

    print("=" * 70)
    print(f"PDF: {pdf_path.name}")
    print(f"chunk_size={size}")
    print(f"chunk_overlap={int(size * 0.1)}")
    print(f"chunk_count={len(chunks)}")
    print()

    # 첫 번째 chunk 정보 확인
    print("[첫 번째 chunk metadata]")
    print(chunks[0].metadata)
    print()

    print("[첫 번째 chunk preview]")
    print(chunks[0].page_content[:500])
    print()

    # 전체 chunk 확인
    print("[전체 chunk 출력]")
    for i, chunk in enumerate(chunks):
        print("-" * 70)
        print(f"{i + 1}번 청크")
        print(f"길이: {len(chunk.page_content)}")
        print(f"metadata: {chunk.metadata}")
        print("내용:")
        print(repr(chunk.page_content))
        print()
```

10_token_example.py

```
import tiktoken
```

```
text = "나는 LangChain을 좋아합니다."
```

```
encoding_cl100k = tiktoken.get_encoding("cl100k_base")
```

```
encoding_o200k = tiktoken.get_encoding("o200k_base")
```

```
print("문장:", text)
```

```
print("cl100k_base token 수:", len(encoding_cl100k.encode(text)))
```

```
print("o200k_base token 수:", len(encoding_o200k.encode(text)))
```

```
print("o200k token ids:", encoding_o200k.encode(text))
```

11_Text Splitter_TokenTextSplitter.py

```
import tiktoken
from langchain_text_splitters import TokenTextSplitter

encoding = tiktoken.get_encoding("o200k_base")

def num_tokens_from_string(string: str) -> int:
    return len(encoding.encode(string))

sample_text = "This splitter is essential for managing LLM context windows and optimizing API costs by counting tokens."

print(f"원본 텍스트의 총 토큰 수: {num_tokens_from_string(sample_text)}\n")

text_splitter = TokenTextSplitter(
    chunk_size=15,
    chunk_overlap=5,
    encoding_name="o200k_base",
)

chunks = text_splitter.split_text(sample_text)

print(f"--- TokenTextSplitter 결과: 총 {len(chunks)}개 ---")
for i, chunk in enumerate(chunks):
    print(f"[{i+1}] (토큰: {num_tokens_from_string(chunk)}) {chunk}")
```

```
import tiktoken
```

```
encoding = tiktoken.get_encoding("o200k_base")
```

```
examples = [  
    "I love LangChain",  
    "나는 랭체인을 좋아합니다",  
    "RAG = Retrieval Augmented Generation",  
]
```

```
for text in examples:
```

```
    tokens = encoding.encode(text)
```

```
    print("=" * 50)
```

```
    print("문장:", text)
```

```
    print("토큰 수:", len(tokens))
```

```
    print("토큰 ID:", tokens)
```

```
    print("토큰 분해:")
```

```
    for token in tokens:
```

```
        print(f'"{encoding.decode([token])}" ({token})")
```

13_Text Splitter_MarkdownHeaderTextSplitter.py

```
from pathlib import Path
from langchain_text_splitters import MarkdownHeaderTextSplitter

markdown_text =
Path("./data/Sample_Markdown.md").read_text(encoding="utf-8")

headers_to_split_on = [
    ("#", "Header 1"),
    ("##", "Header 2"),
    ("###", "Header 3"),
]

splitter =
MarkdownHeaderTextSplitter(headers_to_split_on=headers_to_split_o
n)
chunks = splitter.split_text(markdown_text)

print(f"--- MarkdownHeaderTextSplitter 결과: 총 {len(chunks)}개 ---")
for i, chunk in enumerate(chunks):
    print("=" * 50)
    print(f"chunk {i+1}")
    print("metadata:", chunk.metadata)
    print("page_content:")
    print(chunk.page_content)
```

14_Text Splitter_CodeSplitter.py

```
from pathlib import Path
from langchain_text_splitters import RecursiveCharacterTextSplitter,
Language

python_code =
Path("./data/Sample_Code.py").read_text(encoding="utf-8")

splitter = RecursiveCharacterTextSplitter.from_language(
    language=Language.PYTHON,
    chunk_size=300,
    chunk_overlap=50,
)

chunks = splitter.split_text(python_code)

print(f"--- CodeSplitter 결과: 총 {len(chunks)}개 ---")
for i, chunk in enumerate(chunks):
    print("=" * 50)
    print(f"chunk {i+1}")
    print(chunk)
```

15_Text Splitter_SemanticChunker.py

```
# 설치가 필요합니다: pip install langchain-experimental
from dotenv import load_dotenv
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

load_dotenv()

sample_text = (
    "태양은 별입니다. 달은 지구 주위를 돕니다. "
    "RAG는 외부 문서를 검색해서 답변에 참고하는 방식입니다. "
    "Embedding은 문장의 의미를 숫자 벡터로 바꾸는 과정입니다."
)

splitter = SemanticChunker(OpenAIEmbeddings(model="text-embedding-3-small"))
chunks = splitter.split_text(sample_text)

print(f"--- SemanticChunker 결과: 총 {len(chunks)}개 ---")
for i, chunk in enumerate(chunks):
    print(f"[{i+1}] {chunk}")
```


16_Embedding_OpenAIEmbeddings.py

```
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings
import os

load_dotenv()

embeddings_model =
OpenAIEmbeddings(model=os.getenv("OPENAI_EMBEDDINGS_MODEL", "text-embedding-3-small"))

documents = [
    "저는 강아지를 좋아합니다.",
    "저는 고양이를 좋아합니다.",
    "저는 축구를 좋아합니다.",
]

document_embeddings =
embeddings_model.embed_documents(documents)
query = "제가 좋아하는 동물은 무엇인가요?"
query_embedding = embeddings_model.embed_query(query)

print("문서 embedding 개수:", len(document_embeddings))
print("문서 embedding 차원:", len(document_embeddings[0]))
print("질문 embedding 차원:", len(query_embedding))
print("첫 번째 문서 embedding 앞 5개:",
document_embeddings[0][:5])
```

17_Embedding_PDF_Pages.py

```
from pathlib import Path
from dotenv import load_dotenv
from langchain_community.document_loaders import PyPDFLoader
from langchain_openai import OpenAIEmbeddings
import os

load_dotenv()

spri_pdf = Path("./data/SPRi AI Brief_10월호_산업동향_1002_F.pdf")
sample_pdf = Path("./data/Sample_RAG_Knowledge_Base.pdf")
pdf_path = spri_pdf if spri_pdf.exists() else sample_pdf

loader = PyPDFLoader(str(pdf_path))
documents = loader.load()

embeddings_model =
OpenAIEmbeddings(model=os.getenv("OPENAI_EMBEDDINGS_MODEL
", "text-embedding-3-small"))

sample_docs = documents[:3]
document_texts = [doc.page_content for doc in sample_docs]
document_embeddings =
embeddings_model.embed_documents(document_texts)

print("PDF:", pdf_path.name)
print("embedding 개수:", len(document_embeddings))
print("embedding 차원:", len(document_embeddings[0]))
print("첫 번째 vector 앞 5개:", document_embeddings[0][:5])
```

18_Embedding_similarity_demo.py

```
from __future__ import annotations
import os
import math
from dotenv import load_dotenv
from langchain_openai import OpenAIEmbeddings

load_dotenv()

def cosine_similarity(a: list[float], b: list[float]) -> float:
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(y * y for y in b))
    return dot / (norm_a * norm_b)

documents = [
    "AI 반도체 시장이 빠르게 성장하고 있습니다.",
    "생성형 AI 서비스가 기업 업무에 도입되고 있습니다.",
    "오늘 점심 메뉴는 김치찌개입니다.",
]
query = "인공지능 산업 동향은 어떤가요?"

embeddings =
OpenAIEmbeddings(model=os.getenv("OPENAI_EMBEDDINGS_MODEL", "text-embedding-3-small"))
doc_vectors = embeddings.embed_documents(documents)
query_vector = embeddings.embed_query(query)

scores = []
for doc, vec in zip(documents, doc_vectors):
    score = cosine_similarity(query_vector, vec)
    scores.append((score, doc))

scores.sort(reverse=True)

print("질문:", query)
print("--- 유사도 순위 ---")
for score, doc in scores:
    print(f"{score:.4f} | {doc}")
```